Document Number: P1061R0
Date: 2018-05-01
Audience: EWG
Reply-To: Barry Revzin, barry dot revzin at gmail dot com
Jonathan Wakely, jonathan dot wakely at gmail dot com

Structured Bindings can introduce a Pack

Contents

1. Motivation

Function parameter packs and tuples are conceptually very similar. Both are heterogeneous sequences of objects. Some problems are easier to solve with a parameter pack, some are easier to solve with a `tuple`. Today, it's trivial to convert a pack to a `tuple`, but it's somewhat more involved to convert a `tuple` to a pack. You have to go through `std::apply()`:

```
std::tuple<A, B, C> tup = ...;
std::apply([&](auto&&... elems){
    // now I have a pack
}, tup);
```

This is great for cases where we just need to call a [non-overloaded] function or function object, but rapidly becomes much more awkward as we dial up the complexity. Not to mention if I want to return from the outer scope based on what these elements have to be.

How do we compute the dot product of two `tuple`s? It's a choose your own adventure of awkward choices:

Nested `apply()`

```
template <class P, class Q>
auto dot_product(P p, Q q) {
    return std::apply([&](auto... p_elems){
        return std::apply([&](auto... q_elems){
            return (... + (p_elems * q_elems));
        }, q)
    }, p);
}
```

Using `index_sequence`

```
template <size_t... Is, class P, class Q>
auto dot_product(std::index_sequence<Is...>, P p, Q, q) {
    return (... + (std::get<Is>(p) * std::get<Is>(q)));
}

template <class P, class Q>
auto dot_product(P p, Q q) {
    return dot_product(
        std::make_index_sequence<std::tuple_size<P>::value>{},
        p, q);
}
```

Regardless of which option you dislike the least, both are limited to only `std::tuple`s. We don't have the ability to do this at all for any of the other kinds of types that can be used in a structured binding declaration - because we need to explicit list the correct number of identifiers, and we might not know how many there are.

2. Proposal

We propose to extend the structured bindings syntax to allow the user to introduce a pack as the last identifier, following the usual rules of pack declarations (must be trailing, and packs are introduced with leading `...`):

```
std::tuple<X, Y, Z> f();
auto [x,y,z] = f();          // OK today
auto [...xs] = f();          // proposed: xs is a pack of length three containing an X, Y, and a Z
auto [x, ...rest] = f();     // proposed: x is an X, rest... is a pack of length two
auto [x,y,z, ...rest] = f(); // proposed: rest... is an empty pack
auto [x, ...rest, z] = f();  // ill-formed: non-trailing pack. This is for consistency with, for instance, function parameter packs
```

If we additionally add the structured binding customization machinery to `std::integer_sequence`, this could greatly simplify generic code:

	Today	Proposed

Implementing `std::apply()`	`namespace detail {`
	`  template <class F, class Tuple, std::size_t... I>`
	`  constexpr decltype(auto) apply_impl(F &&f, Tuple &&t,`
	`  std::index_sequence<I...>)`
	`  {`
	`  return std::invoke(std::forward<F>(f),`
	`  std::get<I>(std::forward<Tuple>(t))...);`
	`  }`
	`}`
	
	`template <class F, class Tuple>`
	`constexpr decltype(auto) apply(F &&f, Tuple &&t)`
	`{`
	`  return detail::apply_impl(`
	`  std::forward<F>(f), std::forward<Tuple>(t),`
	`  std::make_index_sequence<std::tuple_size_v<`
	`  std::decay_t<Tuple>>>{});`
	`}`
		`template <class F, class Tuple>`
		`constexpr decltype(auto) apply(F &&f, Tuple &&t)`
		`{`
		`  auto&& [...elems] = t;`
		`  return std::invoke(std::forward<F>(f),`
		`  forward_like<Tuple, decltype(elems)>(elems)...);`
		`}`

`dot_product()`, nested	`template <class P, class Q>`
	`auto dot_product(P p, Q q) {`
	`  return std::apply([&](auto... p_elems){`
	`  return std::apply([&](auto... q_elems){`
	`  return (... + (p_elems * q_elems));`
	`  }, q)`
	`  }, p);`
	`}`
		`template <class P, class Q>`
		`auto dot_product(P p, Q q) {`
		`  // no indirection!`
		`  auto&& [...p_elems] = p;`
		`  auto&& [...q_elems] = q;`
		`  return (... + (p_elems * q_elems));`
		`}`

`dot_product()`, `index_sequence`	`template <size_t... Is, class P, class Q>`
	`auto dot_product(std::index_sequence<Is...>, P p, Q, q) {`
	`  return (... + (std::get<Is>(p) * std::get<Is>(q)));`
	`}`
	
	`template <class P, class Q>`
	`auto dot_product(P p, Q q) {`
	`  return dot_product(`
	`  std::make_index_sequence<std::tuple_size_v<P>>{},`
	`  p, q);`
	`}`
		`template <class P, class Q>`
		`auto dot_product(P p, Q q) {`
		`  // no helper function necessary!`
		`  auto [...Is] = std::make_index_sequence<`
		`  std::tuple_size_v<P>>{};`
		`  return (... + (std::get<Is>(p) * std::get<Is>(q)));`
		`}`

Not only are these implementations more concise, but they are also more functional. I can just as easily use `apply()` with user-defined types as I can with `std::tuple`:

```
struct Point {
    int x, y, z;
};

Point getPoint();
double calc(int, int, int);

double result = std::apply(calc, getPoint()); // ill-formed today, ok with proposed implementation
```

3. Wording

Add a new grammar option for *simple-declaration* to 10 [dcl.dcl]:

> *simple-declaration:*
>
> *decl-specifier-seq init-declarator-list*$_{opt}$;
>
> *attribute-specifier-seq decl-specifier-seq init-declarator-list* ;
>
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ [*identifier-list*] *initializer* ;
>
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ [*identifier-list*$_{opt}$ ...*identifier*] *initializer* ;

Expand 10 [dcl.dcl] paragraph 8:

> A *simple-declaration* with an *identifier-list* or an *identifier* with preceding ellipsis is called a structured binding declaration ([dcl.struct.bind]). The *decl-specifier-seq* shall contain only the *type-specifier* `auto` and *cv-qualifiers*. The *initializer* shall be of the form "= *assignment-expression*", of the form "{ *assignment-expression* }", or of the form "(*assignment-expression*)", where the *assignment-expression* is of array or non-union class type.

Change 11.5 [dcl.struct.bind] paragraph 1:

> A structured binding declaration introduces ~~the identifiers v_0, v_1, v_2, ... of the *identifier-list* as~~ names ([basic.scope.declarative]) of *structured bindings*. If the declaration contains an *identifier-list*, the declaration introduces the identifiers v_0, v_1, v_2, ... of the *identifier-list* as names. If the declaration contains an *identifier* with preceding ellipsis, the declaration introduces a *structured binding pack* ([temp.variadic]). Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq*.

Introduce a new paragraph after [dcl.struct.bind] paragraph 1, introducing the term "structured binding size":

> The *structured binding size* of a type `E` is the required number of names that need to be introduced by the structured binding declaration, as defined below. If there is no structured binding pack, then the number of elements in the *identifier-list* shall be equal to the structured binding size. Otherwise, the number of elements of the structured binding pack is the structured binding size less the number of elements in the *identifier-list*.

Change [dcl.struct.bind] paragraph 2 to define a structured binding size:

> If `E` is an array type with element type `T`, ~~the number of elements in the *identifier-list*~~ the structured binding size of `E` shall be equal to the number of elements of `E`. ~~Each v_i~~ The i^{th} *identifier* is the name of an lvalue that refers to the element *i* of the array and whose type is `T`; the referenced type is `T`.

Change [dcl.struct.bind] paragraph 3 to define a structured binding size:

> Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the ~~number of elements in the *identifier-list*~~ structured binding size of `E` shall be equal to the value of that expression. [...] ~~Each v_i~~ The i^{th} *identifier* is the name of an lvalue of type T_i that refers to the object bound to r_i; the referenced type is T_i.

Change [dcl.struct.bind] paragraph 4 to define a structured binding size:

> Otherwise, all of `E`'s non-static data members shall be direct members of `E` or of the same base class of `E`, well-formed when named as `e.name` in the context of the structured binding, `E` shall not have an anonymous union member, and the ~~number of elements in the *identifier-list*~~ structured binding size of `E` shall be equal to the number of non-static data members of `E`. Designating the non-static data members of `E` as $m_0, m_1, m_2, \ldots$ (in declaration order), ~~each v_i~~ the i^{th} *identifier* is the name of an lvalue that refers to the member m_i of `e` and whose type is *cv* T_i, where T_i is the declared type of that member; the referenced type is *cv* T_i. The lvalue is a bit-field if that member is a bit-field.

Add a new clause to 17.6.3 [temp.variadic], after paragraph 3:

> A *structured binding pack* is an *identifier* that introduces zero or more *structured bindings* ([dcl.struct.bind]). *[Example* ```
> auto foo() -> int(&)[2];
> auto [...a] = foo(); // a is a structured binding pack containing 2 elements
> auto [b, c, ...d] = foo(); // d is a structured binding pack containing 0 elements
> auto [e, f, g, ...h] = foo(); // error: too many identifiers
> ``` *-end example]*

In 17.6.3 [temp.variadic], change paragraph 4:

> A *pack* is a template parameter pack, a function parameter pack, ~~or~~ an *init-capture* pack, or a structured binding pack. The number of elements of a template parameter pack or a function parameter pack is the number of arguments provided for the parameter pack. The

> number of elements of an *init-capture* pack is the number of elements in the pack expansion of its *initializer*.

In 17.6.3 [temp.variadic], paragraph 5 (describing pack expansions) remains unchanged.

In 17.6.3 [temp.variadic], add a bullet to paragraph 8:

> Such an element, in the context of the instantiation, is interpreted as follows:
> - if the pack is a template parameter pack, the element is a template parameter ([temp.param]) of the corresponding kind (type or non-type) designating the i^{th} corresponding type or value template argument;
> - if the pack is a function parameter pack, the element is an *id-expression* designating the i^{th} function parameter that resulted from instantiation of the function parameter pack declaration; otherwise
> - if the pack is an *init-capture* pack, the element is an *id-expression* designating the variable introduced by the i^{th}th *init-capture* that resulted from instantiation of the *init-capture* pack~~.~~ ; otherwise
> - if the pack is a structured binding pack, the element is an *id-expression* designating the i^{th} structured binding that resulted from the structured binding declaration.

4. Acknowledgements

Thanks to Michael Park and Tomasz Kamiński for their helpful feedback.

5. References

- [N3915] "apply() call a function with arguments from a tuple (V3)" by Peter Sommerlad, 2014-02-14
- [P0144] "Structured Bindings" by Herb Sutter, 2016-03-16